

module.

Summary

The Tower of Hanoi puzzle game and Four-in-a-Row game are short programs, but by following the practices in this book, you can ensure that their code is readable and easy to debug. These programs follow several good practices: they've been automatically formatted with Black, use docstrings to describe the module and functions, and place the constants near the top of the file. They limit the variables, function parameters, and function return values to a single data type so type hinting, although a beneficial form of additional documentation, is unnecessary.

In the Tower of Hanoi, we represent the three towers as a dictionary with keys 'A', 'B', and 'C' whose values are lists of integers. This works, but if our program were any larger or more complicated, it would be a good idea to represent this data using a class. Classes and OOP techniques weren't used in this chapter because I don't cover OOP until Chapters 15 through 17. But keep in mind that it's perfectly valid to use a class for this data structure. The towers render as ASCII art onscreen, using text characters to show each disk of the towers.

The Four-in-a-Row game uses ASCII art to display a representation of the game board. We display this using a multiline string stored in the `BOARD_TEMPLATE` constant. This string has 42 brace pairs `{}` to display each space on the 7 by 6 board. We use braces so the `format()` string method can replace them with the tile at that space. This way, it's more obvious how the `BOARD_TEMPLATE` string produces the game board as it appears onscreen.

Although their data structures differ, these two programs share many similarities. They both render their data structures onscreen, ask the player for input, validate that input, and then use it to update their data structures before looping back to the beginning. But there are many different ways we could have written code to carry out these actions. What makes code readable is ultimately a subjective opinion rather than an objective measure of how closely it adheres to some list of rules. The source code in this chapter shows that although we should always give any code smells a second look, not all code smells indicate a problem that we need to fix.